Local Search Algorithms

AI optimization when the path is not the solution

FROM SEARCHING PATHS → IMPROVING CONFIGURATIONS

Hill Climbing • Random Restart • Simulated Annealing • Beam Search • Genetic Algorithms • Continuous Optimization



Learning objectives

By the end of the lecture

- Explain why local search treats the goal as a complete configuration.
- Define state, neighbor, objective / evaluation function, and local optimum.
- Implement hill climbing and trace every step in Python.
- Diagnose local maxima, plateaus, and ridges.
- Explain why random restarts and simulated annealing help.
- Compare local beam search, genetic algorithms, and continuous optimization.

Where local search fits

From A* to optimization

A* / UCS

Search for a path. Cost accumulates along the route.

LOCAL SEARCH

Path is often irrelevant; the final configuration is what matters.

OPTIMIZATION

Find a configuration that minimizes or maximizes an objective.

Key shift

Keep one or a few candidate states and improve them instead of retaining a large search frontier.

Why use local search?

When the path is irrelevant

- Many problems ask for a complete configuration, not a route to reach it.
- Examples: n-queens and travelling salesperson problem.
- Local search can use constant space.

Local search is a family of optimization algorithms that start with one possible solution and try to improve it by moving to a better neighboring solution.
Unlike BFS, DFS, UCS, and A*, local search usually does not care about the complete path used to reach the solution.

MEMORY PROFILE

Keep a current state instead of the entire explored region.

Can I improve my current solution by making a small change?

The local-search state space

A state is a complete candidate solution

STATE

One complete configuration.

NEIGHBOR

A small change to the current configuration.

VALUE

How good is this configuration?

Current state → generate neighbors → evaluate → choose next state

Objective functions

Maximize or minimize

MAXIMIZATION

Higher value is better.
Example: maximize throughput or fitness.

MINIMIZATION

Lower value is better.
Example: minimize route length or conflicts.

Design question

Can we express the quality of a configuration as one measurable number?

Neighborhoods

What counts as a one-step move?

8-PUZZLE

Move the blank tile to an adjacent position.

N-QUEENS

Move one queen within its column.

TSP

Swap or reverse part of a route.

The neighborhood definition changes the algorithm's behavior.

Hill climbing

The simplest iterative improvement algorithm

- Start from an initial state.
- Generate neighboring states.
- Move to a better neighbor.
- Repeat until no neighbor improves the current value.

Current → better neighbor → better neighbor → **stop**

Hill climbing: step by step

A maximization example

```
values = [1, 3, 5, 8, 6, 4, 2]
start = 0
```

STEP 0

index 0
value 1

STEP 1

index 1
value 3

STEP 2

index 2
value 5

STEP 3

index 3
value 8

The algorithm keeps moving uphill until no adjacent value is better.

Python: hill climbing skeleton

The core loop

```
def hill_climbing(values, start):
    current = start
    while True:
        neighbors = get_neighbors(current)
        best = choose_best(neighbors)
        if value(best) <= value(current):
            return current
        current = best
```

1. CURRENT

Where am I now?

2. NEIGHBORS

What can I change?

3. STOP

No improvement.

Python: concrete implementation

No hidden magic — inspect every operation

```
def hill_climbing(values, start):
    current = start
    while True:
        neighbors = []
        if current > 0:
            neighbors.append(current - 1)
        if current < len(values) - 1:
            neighbors.append(current + 1)
        best = current
        for n in neighbors:
            if values[n] > values[best]:
                best = n
        if best == current:
            break
        current = best
    return current
```

INDEX

current is a position in the list.

NEIGHBORS

left = current - 1
right = current + 1

DECISION

Move only when value improves.

Trace the Python execution

values = [1, 3, 5, 8, 6, 4, 2]

START

current = 0
value = 1

MOVE

1 → 3

MOVE

3 → 5

MOVE

5 → 8

At 8, neighbors are 5 and 6. Neither improves the value.

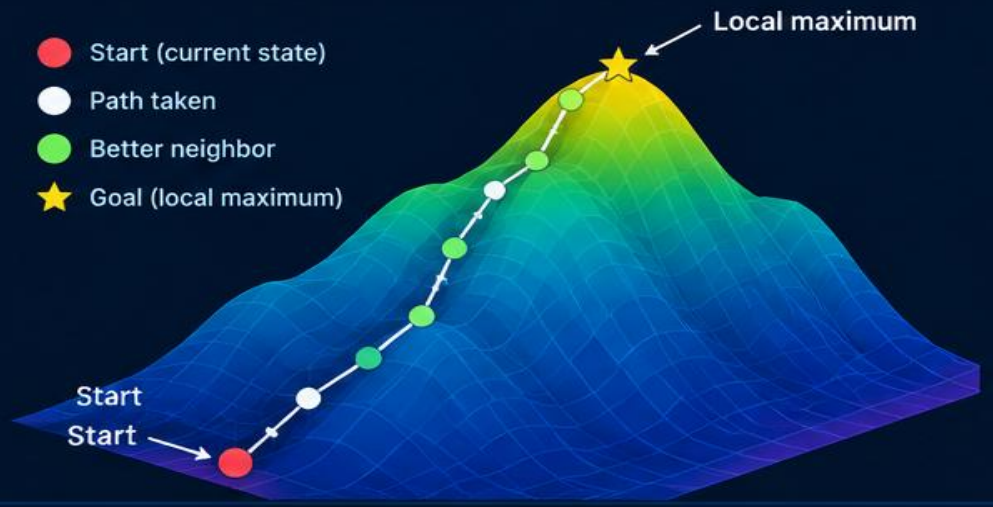
RETURN 8

Hill Climbing Algorithm

A simple local search method that repeatedly moves to a better neighboring state until no improvement is found.

Core Idea

Start from a solution, look at its neighbors, move to a better one, and repeat.



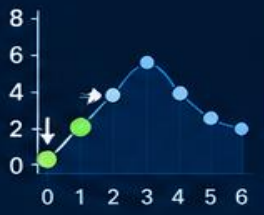
Step-by-Step Example

1 Start



Current state: index 0 (value = 1)

2 Move to better neighbor



Move to index 1 (value = 3)

3 Continue climbing



Move to index 2 (value = 5)

4 Reach the peak



Move to index 3 (value = 8)

5 Stop



No better neighbor found. Algorithm stops.

Pseudocode

```
def hill_climbing(landscape, start):
    current = start
    while True:
        neighbors = get_neighbors(current)
        best = curnent
        for n in neighbors:
            if landscape[n] > landscape[best]:
                best = n
        if best == current: # no improvement
            break
        current = best

    return current
```

Python Example

```
landscape = [1, 3, 5, 8, 6, 4, 2]
start = 0
def hill_climbing(landscape, start):
    current = start
    while True:
        neighbors = []
        if current > 0:
            neighbors.append(current - 1)
        if current < len(landscape) - 1:
            neighbors.append(current + 1)
        best = current
        for n in neighbors:
            if landscape[n] > landscape[best]:
                best = n
        if best == current:
            break
        current = best
    return current
```

Output (trace)

```
Start at index 0 (value = 1)
Move to index 1 (value = 3)
Move to index 2 (value = 5)
Move to index 3 (value = 8)
Stop at index 3 (local maximum)
```

Landscape



Key Points

Simple
Easy to understand and implement.

Fast
Usually requires few evaluations.

Limitations
- Gets stuck in local maxima
- Struggles with plateaus and ridges

Variants
- Steepest-ascent
- Random-restart
- Simulated annealing (accepts worse moves)

Local maximum

Good nearby ≠ globally best



Hill climbing can stop at a peak that is higher than its neighbors but lower than another peak.

Plateaus and ridges

Two more failure modes

PLATEAU

Many states have the same value.
The algorithm has little gradient information and may stop or wander.

RIDGE

The best direction may require a sequence of moves that is not directly available as a single improving neighbor.

Diagnosis matters: the landscape is part of the algorithm design.

Random-restart hill climbing

Escape dependence on the initial state

RUN 1 start → local peak

RUN 2 start → local peak

RUN 3 start → global peak ✓

STRATEGY

Sample several starting states.
Run hill climbing from each.
Return the best result found.

Trade-off: more computation for less sensitivity to one unlucky start.

Python: random restart

A simple wrapper around local search

```
import random

for _ in range(attempts):
    start = random.randint(0, len(values)-1)
    position = hill_climbing(values, start)
    keep_best(position)
```

RANDOMIZE

Different initial states.

REPEAT

Independent improvement runs.

SELECT

Keep best result.

Why random restart works conceptually

Multiple starts sample multiple basins



Different initial states can converge to different local optima.

Random-Restart Hill Climbing

Run hill climbing multiple times from different random starting points
to avoid getting stuck in local maxima.

Key Idea

Each run may end at a local maximum,
but by restarting from a new random point,
we increase the chance of finding the global maximum.



How It Works

1. Choose a random starting state.
2. Run hill climbing until no better neighbor is found (local maximum).
3. Restart from a new random state.
4. Keep the best solution found across all runs.

Python Example

```python
import random
def random_restart_hill_climbing(values, runs=10):
    best_pos = None
    best_val = float('-inf')
    for _ in range(runs):
        start = random.randint(0, len(values) - 1)
        pos = hill_climbing(values, start)
        if values[pos] > best_val:
            best_val = values[pos]
            best_pos = pos
    return best_pos, best_val
```

Example Run

Run	Start	Local Max	Value
1	2	3	8
2	0	2	6
3	5	6	7
4	1	3	8
5	4	6	7

Best solution: index 3 (value = 8)

Why It Works

- Simple to implement
- Helps escape local maxima
  (by exploring different starting points)
- Often finds better solutions
  than a single run
- Useful for complex, multi-modal
  landscapes

Simulated annealing

Sometimes accept a worse move

- Like hill climbing, it explores neighboring states.
- Unlike hill climbing, it can accept a worse move.
- Acceptance is controlled by a temperature parameter.
- Temperature decreases over time.

HIGH T **explore** → LOW T **refine**

Simulated Annealing

- Theoretical guarantee:
  - Stationary distribution (Boltzmann): $P(x) \propto e^{E(x)/T}$
  - If T decreased slowly enough, will converge to optimal state!

- Proof sketch
  - Consider two adjacent states x, y with $E(y) > E(x)$ [high is good]
  - Assume $x \longrightarrow y$ and $y \longrightarrow x$ and outdegrees $D(x) = D(y) = D$
  - Let $P(x)$, $P(y)$ be the equilibrium occupancy probabilities at T
  - Let $P(x \longrightarrow y)$ be the probability that state x transitions to state y



Temperature controls exploration

A schedule, not a single magic number

HIGH TEMPERATURE

Bad moves are more likely.
The search can cross valleys.

COOLING

Temperature decreases according to a schedule.

LOW TEMPERATURE

Behavior becomes increasingly greedy.

Explore early; exploit later.

Simulated annealing: acceptance rule

Maximization form

```
ΔE = next.value - current.value

if ΔE > 0:
    accept next
else:
    accept with probability exp(ΔE / T)
```

BETTER MOVE

Always accepted.

WORSE MOVE

Sometimes accepted.

COOLING

Worse moves become less likely.

Cooling schedules

Why schedule design matters

- Too fast: the search can freeze near a poor configuration.
- Too slow: the algorithm may require excessive computation.
- The theoretical convergence result requires sufficiently slow cooling.

PRACTICAL QUESTION

How much exploration do we need before we start trusting local improvement?

Python: simulated annealing

Compact implementation

```
import random, math

def anneal(values, start, T=10):
    current = start
    while T > 0.1:
        next_state = random_neighbor(current)
        dE = values[next_state] - values[current]
        if dE > 0 or random.random() < math.exp(dE/T):
            current = next_state
        T *= 0.95
    return current
```

ΔE

Improvement if positive.

exp(ΔE/T)

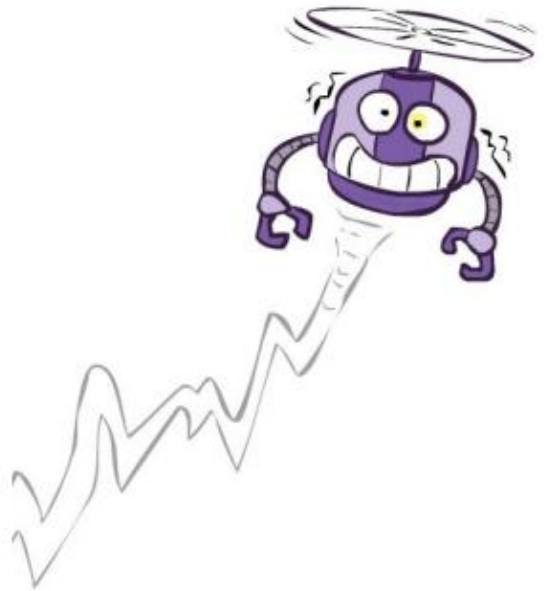
Controls bad-move probability.

T *= 0.95

Cooling schedule.

Simulated Annealing

- Is this convergence an interesting guarantee?
- Sounds like magic, but reality is reality:
  - The more downhill steps you need to escape a local optimum, the less likely you are to ever make them all in a row
  - "Slowly enough" may mean exponentially slowly
  - Random restart hill climbing also converges to optimal state…
- Simulated annealing and its relatives are a key workhorse in VLSI layout and other optimal configuration problems

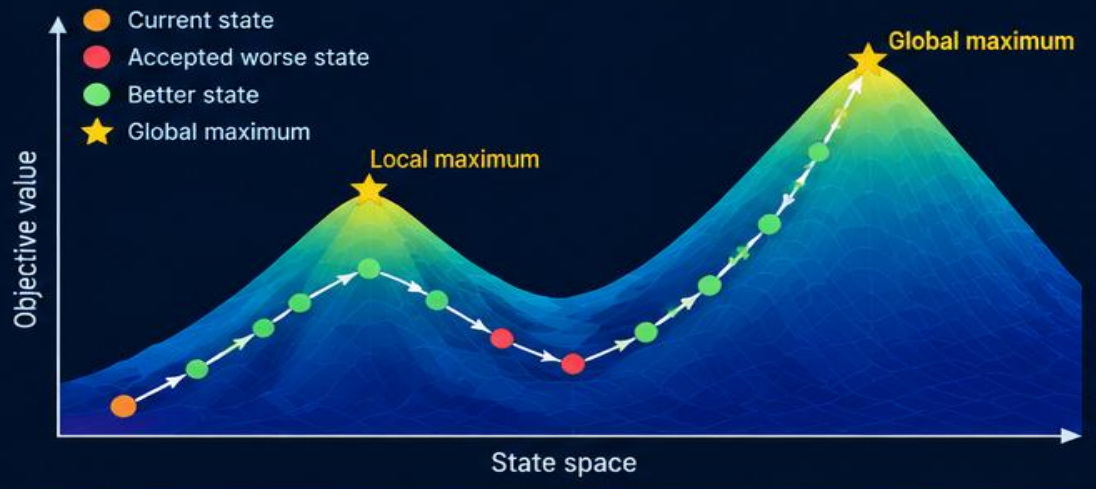


Simulated Annealing

Simulated annealing is a local search algorithm that can accept worse solutions with a certain probability, helping to escape local maxima and find **better solutions.**

Key Idea

Sometimes move to a worse state (especially at high temperature), with decreasing probability over time.



How It Works

1 Initialize



Start from a random state (solution).

2 Generate neighbor



Pick a neighboring state (e.g., slight modification).

3 Decide



If better → accept. If worse → accept with probability $\exp(-\Delta/T)$.

4 Cool down



Decrease temperature (T) over time.

5 Repeat



Continue until T is low or no improvement is found.

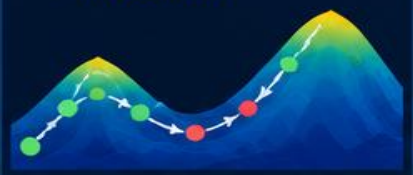
Python Example

```python
import random, math
def simulated_annealing(values, start, T=10, alpha=0.95, min_T=0.1):
    current = start
    best = start
    while T > min_T:
        # pick a random neighbor (left or right)
        neighbors = []
        if current > 0: neighbors.append(current - 1)
        if current < len(values) - 1: neighbors.append(current + 1)
        next_state = random.choice(neighbors)
        delta = values[next_state] - values[current]
        if delta > 0 or random.random() < math.exp(delta / T):
            current = next_state
            if values[current] > values[best]:
                best = current

        T *= alpha  # cool down
    return best
```

Temperature Effect

High Temperature (exploration)



- More likely to accept worse moves
- Explores more of the state space
- Helps escape local maxima

Low Temperature (exploitation)



- Rarely accepts worse moves
- Focuses on improvements
- Fine-tunes the solution

High T (explore) → **Low T** (exploit)

Why Accept a Worse State?



- It may lead to a better solution later!

Key Takeaways

- Can escape local maxima
- Controlled by temperature
- Trade-off between exploration and exploitation
- Widely used in optimization problems

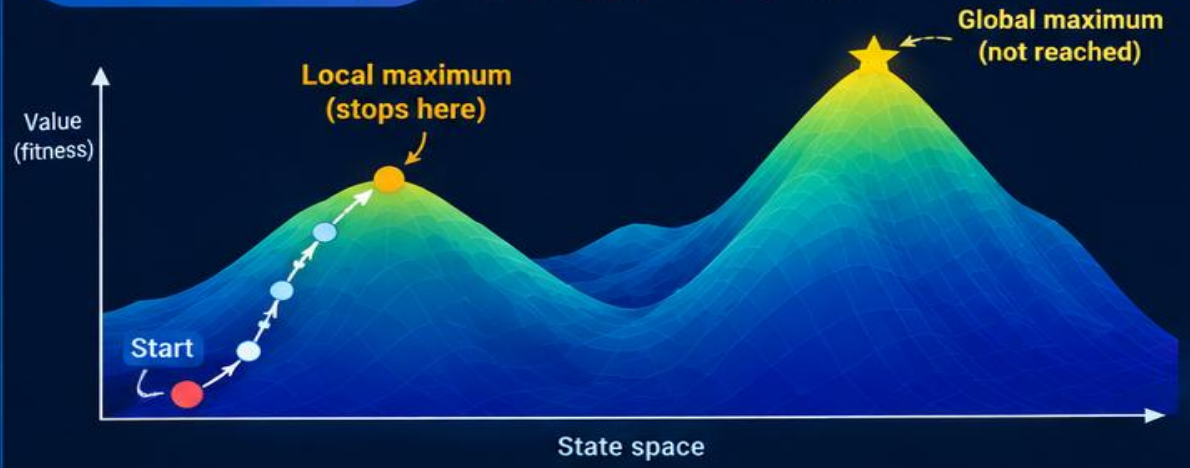
Hill Climbing vs. Simulated Annealing

Both are local search algorithms, but they behave differently when facing local maxima.

Same goal:
find a good (ideally global) maximum by improving the current solution.

Hill Climbing

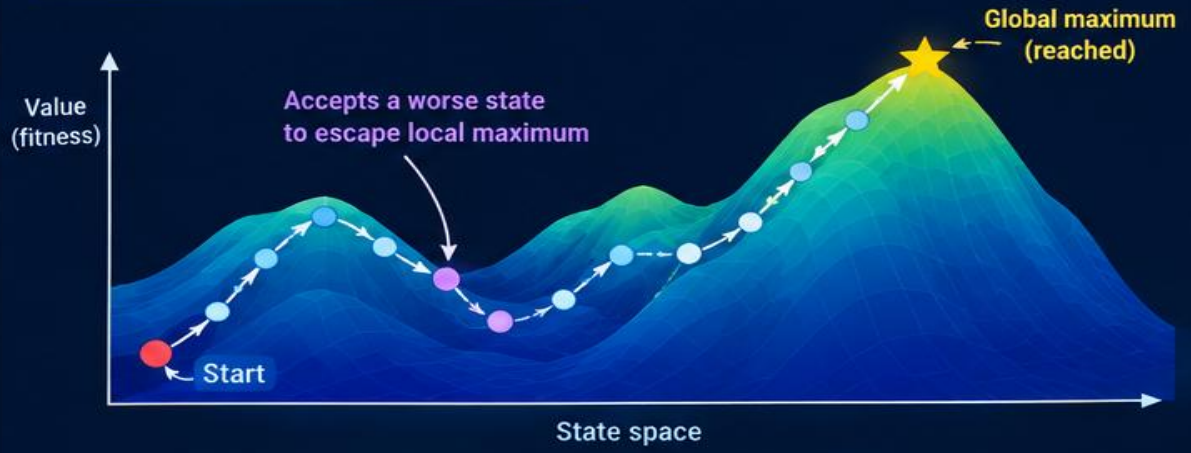
Always moves to a better neighbor.
Never accepts a worse state.



Risk: gets stuck in local maxima, plateaus, or ridges.

Simulated Annealing

Can accept worse states (with a probability), especially at higher temperature.



Advantage: can escape local maxima and find better solutions.

Feature	Hill Climbing	Simulated Annealing
Movement rule	Only moves to a better neighbor	Can move to worse neighbor (with probability)
Local maxima	Gets stuck	Can escape (due to random worse moves)
Temperature	Not used	Uses a decreasing temperature schedule
Exploration vs. exploitation	More exploitation	More exploration early, more exploitation later
Convergence	Fast, but may be suboptimal	Slower, but higher chance of global optimum
Best for	Simple problems, smooth landscapes	Complex landscapes, large search spaces

In short...

Hill Climbing
Simple, fast, but shortsighted.

Simulated Annealing
Smarter exploration,
better chance to find
the global best solution.

Local search + strategy = better results

Local beam search

Keep K candidate states

- Initialize K copies of a local search algorithm, often from random states.
- Generate all successors of the K current states.
- Choose the best K successors as the next set.
- The K searches communicate: a strong region can attract other candidates.

K = 1 → local search

K > 1 → interacting beam

Beam search: why communication matters

Not simply K independent hill climbs

SEARCH A

Finds region X.

SEARCH B

Finds region Y.

SHARED SELECTION

Top K states are pooled and reselected.

The population can shift toward a promising region discovered by another search.

Local Beam Search

Local beam search is a local search algorithm that **keeps multiple candidate states (beam)** at each step, exploring the most promising states in parallel.

Key Idea

At each iteration, expand the k best states from the current beam and keep the best k states for the next round.

How It Works (Step by Step)

1 Initialize

Start with k random (or given) states.



Beam (k = 3)

2 Expand Neighbors

Generate neighbors for each state in the beam.



Each state → neighbors

3 Evaluate & Select

Compute the value (e.g., heuristic) and keep the top k states from all neighbors.



Top k states become the new beam

4 Repeat

Use the new beam and continue the process.



Beam (k = 3)

5 Goal Reached

Stop when a goal state is found (or after a limit of iterations).



Solution

Python Example

```python
import random
def local_beam_search(start_states, k, expand, goal_test, value_fn, max_iter=50):
    beam = list(start_states)
    for _ in range(max_iter):
        # 1. Expand neighbors for all states in beam
        candidates = []
        for state in beam:
            candidates.extend(expand(state))
        # 2. Select top k states
        candidates.sort(key=value_fn, reversse=True)
        beam = candidates[:k]
        # 3. Check for goal
        for state in beam:
            if goal_test(state):
                return state
    return None
```

Example Components

expand(state)
Returns neighbors of a state.

goal_test(state)
Checks if the state is a goal.

value_fn(state)
Heuristic or evaluation function (higher is better).

k
Beam width – number of states kept at each step.

Example: Beam Search on a Mountain



Advantages & Limitations

Advantages

- Explores multiple states in parallel
- Less likely to get stuck in local maxima
- Works well for large search spaces
- Can be combined with different heuristics

Limitations

- Higher memory requirements (stores k states)
- May still miss the global optimum
- Performance depends on beam width k
- Needs a good evaluation function

Local beam search = multiple paths, better chances.

Genetic algorithms

Optimization through a population

- Maintain a population of candidate solutions.
- Select candidates using a fitness function.
- Combine pairs with crossover operators.
- Use mutation to introduce variation.
- Repeat across generations.

SELECTION → CROSSOVER → MUTATION → NEW POPULATION

Genetic algorithm pipeline

A population-based search

POPULATION

candidate states

FITNESS

measure quality

SELECTION

resample

CROSSOVER

recombine

MUTATION

add variety

Repeat until a stopping condition is met.

Genetic Algorithms

Genetic algorithms (GA) are population-based search algorithms inspired by natural evolution. They maintain a set of candidate solutions and iteratively improve them using selection, crossover, and mutation.

Key Idea

Use **evolution** principles — selection, **crossover** and **mutation** — to evolve better solutions over generations.

The Genetic Algorithm Process

1 Initialize Population

010011
110010
001101
101110
011001
...

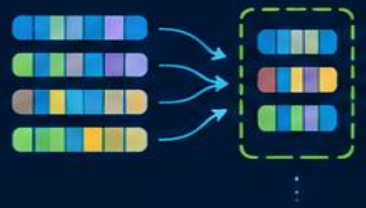
Create a set of random solutions (chromosomes).

2 Evaluate Fitness

	Fitness
	25
	17
	32
	21
	28
...	⋮

Measure how good each solution is using a fitness function.

3 Selection



Choose better solutions for reproduction (e.g., tournament, roulette wheel).

4 Crossover



Combine parts of two parents to create new solutions.

5 Mutation



Introduce small random changes to maintain diversity.

6 New Generation

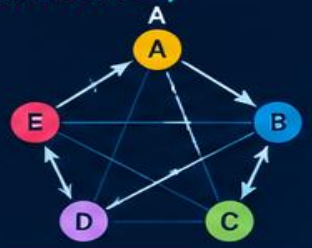


Repeat the process for many generations until a stopping criterion is met.

Example: TSP (Traveling Salesman Problem)

Chromosome representation





Fitness Function

Total distance (smaller is better)

$$f = \frac{1}{1 + \text{distance}}$$

Why It Works

- ✓ Explores multiple solutions
- ✓ Avoids local optima
- ✓ Works well for complex spaces

Genetic Algorithm in Python (Simplified)

```python
import random
def fitness(route, dist):
    return 1 / (1 + dist(route))

for generation in range(100):
    # 1. Selection (tournament)
    parents = select(population, fitness)
    # 2. Crossover
    children = crossover(parents)
    # 3. Mutation
    children = mutate(children, rate=0.1)
    population = children
    # Keep the best
    population = select_next_generation(population)
```

Full code in lab session →

Key Parameters

Population size	Number of candidate solutions
Crossover rate	Probability of recombination (typically 0.6–0.9)
Mutation rate	Probability of random change (typically 0.01–0.1)
Selection method	How parents are chosen
Termination	Max generations or target fitness

Strengths and Limitations

Strengths

- ✓ Good for complex, large spaces
- ✓ Less likely to get stuck in local optima
- ✓ Flexible and parallelizable

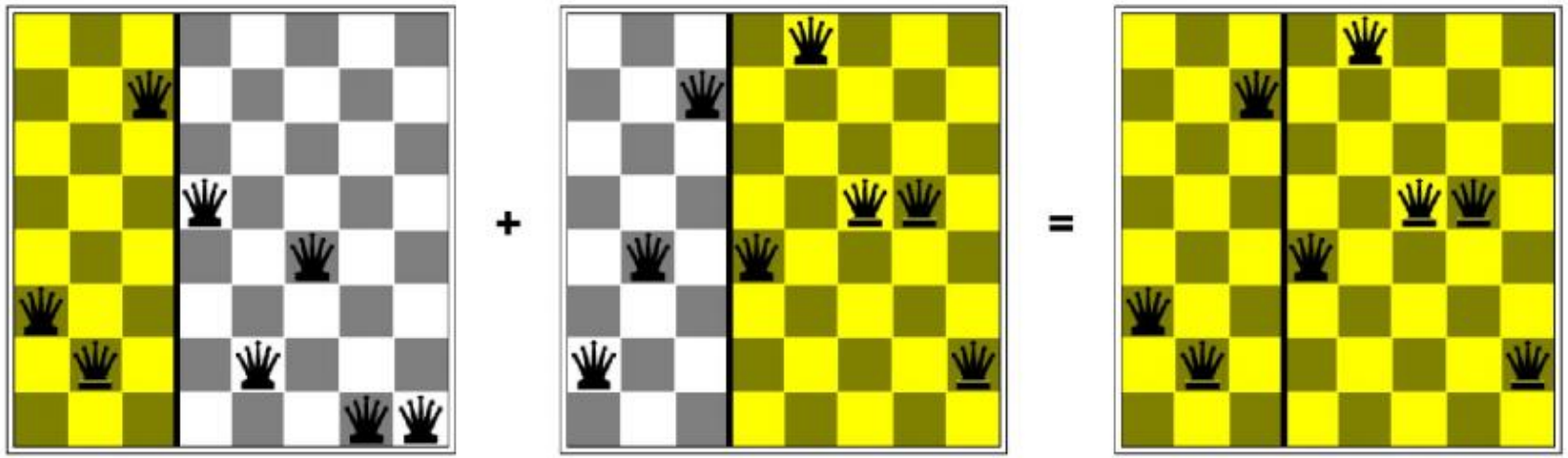
Limitations

- ✗ Slower convergence
- ✗ Requires parameter tuning
- ✗ No guarantee of optimality

Inspiration from nature. Power for optimization.

Better solutions through evolution.

Example: N-Queens



- Does crossover make sense here?
- What would mutation be?
- What would a good fitness function be?

N-Queens as a local-search problem

A canonical configuration problem

STATE

n queens on the board, one queen per column.

ACTION

Move one queen within its column.

VALUE

Number of queen conflicts.

Goal: reach a configuration with zero conflicts.

N-Queens: designing the heuristic

Turn constraints into a measurable landscape

```
def conflicts(board):
    # count pairs of queens that attack
    # each other
    return total_conflicts

# minimization:
# fewer conflicts = better
```

GOOD STATE

few conflicts

GOAL STATE

0 conflicts

NEIGHBOR

move one queen

Traveling Salesperson Problem (TSP)

Local search over complete tours

A → B → C → D → A



A → C → B → D → A

NEIGHBOR MOVE

Swap two cities, or reverse a route segment.

OBJECTIVE

Minimize total tour length.

Python: TSP neighbor generation

A small change creates a new candidate

```
def get_neighbor(route):
    new_route = route.copy()
    i, j = random.sample(range(len(route)),
2)
    new_route[i], new_route[j] =
new_route[j], new_route[i]
    return new_route
```

COPY

Keep the current solution intact.

SWAP

Change two positions.

EVALUATE

Compare route lengths.

Traveling Salesperson Problem (TSP)

The TSP asks for the shortest possible tour that visits each city exactly once and returns to the starting city.

Key Idea

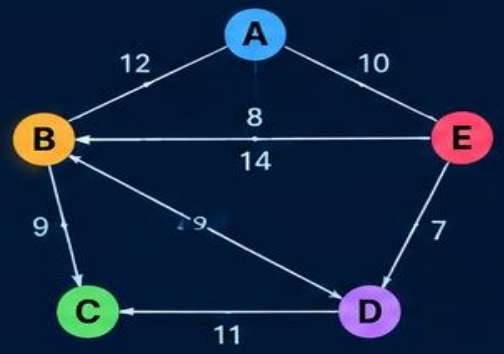
TSP is a combinatorial optimization problem. Local search algorithms can efficiently find good (and sometimes optimal) tours.

Problem Formulation

Cities
n cities, with distances between each pair.

Solution
A permutation of cities forming a cycle.

Objective
Minimize total distance (and return to the start).



Example Tour

A → B → C → D → E → A

Total distance = 12 + 9 + 11 + 7 + 10 = 49

Each city is visited once and we return to the start.

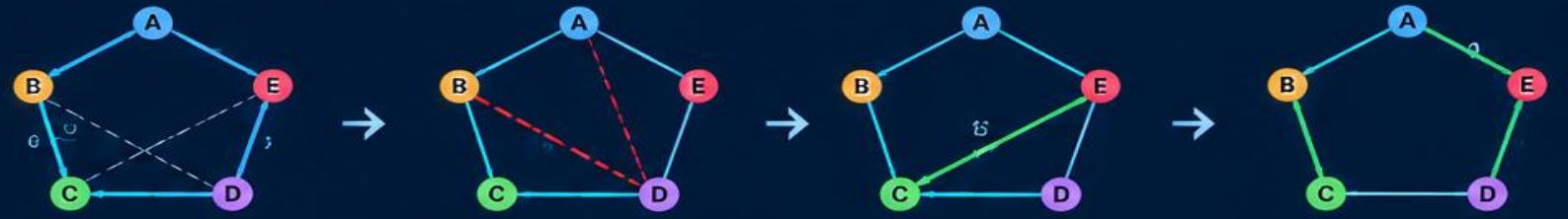
Why It's Challenging?

n cities → (*n* − 1)! /2 possible tours (grows very fast!)

Exact solution is expensive for large *n*.

Local search finds good solutions without checking all tours.

Local Search for TSP – 2-Opt Neighbor Example



Initial tour
A → B → C → D → E → A
(distance = 49)

Choose two edges to swap
Remove (B, C) and (D, E)

Reconnect (2-opt)
Add (B, D) and (C, E)

New tour (better)
A → B → D → C → E → A
(distance = 45)

Python Example: 2-Opt Neighbor

```python
def two_opt(route, dist_matrix):
    n = len(route)
    best_route = route[:]
    best_dist = route_distance(route, dist_matrix)

    for i in range(1, n-2):
        for j in range(i+1, n-1):
            new_route = route[:]
            new_route[i:j+1] = reversed(route[i:j+1])
            new_dist = route_distance(new_route, dist_matrix)
            if new_dist < best_dist:
                best_dist = new_dist
                best_route = new_route
    return best_route, best_dist
```

Additional Heuristics

Nearest Neighbor
Fast, simple, produces a good initial tour.

2-Opt / 3-Opt
Improves a tour by removing and reconnecting edges.

Insertion Heuristic
Builds a tour by adding cities one by one.

Visual Comparison

Initial tour (distance = 49)



After 2-opt (distance = 45)



Key Takeaways

- TSP is a hard combinatorial problem (with factorial complexity).
- Local search (e.g., 2-opt) is effective for finding good solutions.
- Small changes to a tour can lead to significant improvements.
- TSP has many real-world applications: logistics, routing, scheduling, and more.

Local search doesn't guarantee the optimal tour, but it finds high-quality solutions efficiently.

Local search in continuous spaces

States can be real-valued

- Discrete local search changes one combinatorial component at a time.
- Continuous optimization changes numerical variables.
- One approach is to discretize the space and reuse discrete methods.
- Another is random perturbation, as in hill climbing or simulated annealing.
- A third family uses gradients and local curvature.

Example: place facilities / airports to minimize a distance-based objective.

Continuous optimization: perturbations

Small moves in x-space

$$x \rightarrow x + \delta \rightarrow x + \delta'$$

RANDOM PERTURBATION

Propose nearby numerical states and keep moves that improve the objective.

DISCRETIZATION

Create a grid with increment δ, then apply a discrete local-search method.

Gradient descent

Use the slope of the objective

$\nabla f(x)$

$$x \leftarrow x - \alpha \nabla f(x)$$

GRADIENT

Direction of greatest increase.

α

Step size / learning rate.

MINIMIZATION

Move opposite the gradient.

At an extremum, the gradient is zero.

Local Search in Continuous Spaces

In continuous spaces, local search explores nearby points in a continuous domain to find a good solution, typically by improving an objective function.

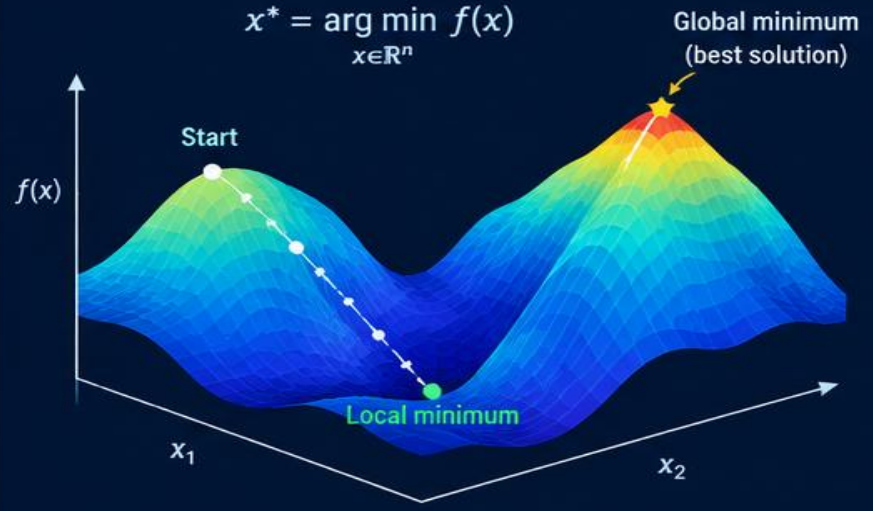
Key Idea

Start from an initial point, evaluate nearby points, and move to a better one. Repeat until no improvement is found (or a stopping criterion is met).

The Problem

Find the minimum (or maximum) of a continuous function:

$$x^* = \arg\min_{x \in \mathbb{R}^n} f(x)$$



Local Search Algorithm (Continuous)

1. **Start**
   Choose an initial point x_0.
2. **Explore Neighbors**
   Sample points in the neighborhood (e.g., random or grid).
3. **Evaluate**
   Compute $f(x)$ for each neighbor.
4. **Move**
   Go to the best improving point (according to the objective).
5. **Repeat**
   Until no improvement or max iterations.

Python Example: Hill Climbing in 2D

```python
import numpy as np
import random

def f(x, y):
    return (x - 2)**2 + (y - 3)**2 + np.sin(x) * np.cos(y)

def local_search(start, step=0.5, max_iter=100):
    x, y = start
    for _ in range(max_iter):
        # Sample nearby points
        neighbors = [(x + dx, y + dy) for dx in [-step, 0, step]
                     for dy in [-step, 0, step] if not (dx == 0 and dy == 0)]

        # Choose the best neighbor
        best = min(neighbors, key=lambda p: f(*p))
        if f(best[0], best[1]) < f(x, y):
            x, y = best
        else:
            break
    return x, y, f(x, y)
```

Visualization: Search Process



Begin at a random point in the search space.

Sample nearby points and evaluate $f(x)$.

Move to the best improving neighbor.

Stop when no improvement is found.

Common Techniques

- **Random Search**
  Sample points randomly in the space.
- **Grid Search**
  Explore fixed grid of points.
- **Gradient Descent**
  Use gradient information to move downhill.
- **Coordinate Search**
  Optimize one dimension at a time.

Example Applications

- Hyperparameter tuning (ML models)
- Function optimization (engineering, science)
- Robotics (continuous control)
- Energy systems (e.g., optimal dispatch)

Local search in continuous spaces is simple, flexible, and powerful — but it can get stuck in local optima.

Airport siting example

A continuous optimization problem

- Choose locations for 3 airports.
- Each city is assigned to its nearest airport.
- Objective: minimize the sum of squared distances from cities to their nearest airport.
- Airport coordinates are continuous variables.

This shows that local search is not limited to grids or discrete puzzles.

Choosing the right local-search strategy

A practical decision map

GREEDY + SIMPLE

Hill climbing
When local improvement is cheap.

MANY STARTS

Random restart
When initial state matters.

ESCAPE VALLEYS

Simulated annealing
When local traps are common.

MULTIPLE STATES

Local beam search

POPULATION

Genetic algorithms

Local search vs A*

Two different problem formulations

A*

State graph + path cost

Keeps frontier / explored information

Goal: reach a goal optimally under conditions

Typical question: “Which path?”

LOCAL SEARCH

Complete configurations

Keeps one or a few candidates

Goal: improve objective value

Typical question: “Which configuration?”

The right algorithm follows from the problem representation.

Practical Python lab

Experiment with the landscape

- Create a 1D landscape such as [1, 3, 5, 8, 6, 4, 2].
- Implement hill climbing and print every state visited.
- Change the starting index and observe different outcomes.
- Add random restarts and compare the best result.
- Implement simulated annealing and visualize accepted moves.
- Extend the same ideas to a TSP route.

Recommended visualization: plot objective value vs iteration.

Key takeaways

What should you remember?

- Local search optimizes complete configurations when the path is not the main concern.
- Hill climbing repeatedly moves to a better neighbor.
- Local maxima, plateaus, and ridges can trap greedy search.
- Random restarts reduce dependence on one initial state.
- Simulated annealing trades greediness for controlled exploration.
- Beam search, genetic algorithms, and gradient methods extend the idea.

Summary + discussion

Local search is a family of optimization strategies

STATE → **NEIGHBORS** → **EVALUATE** → **IMPROVE**

QUESTION 1

When is the path irrelevant?

QUESTION 2

How can we escape a local optimum?

QUESTION 3

How does the neighborhood define the search?

Next step: implement the algorithms and visualize the search dynamics.